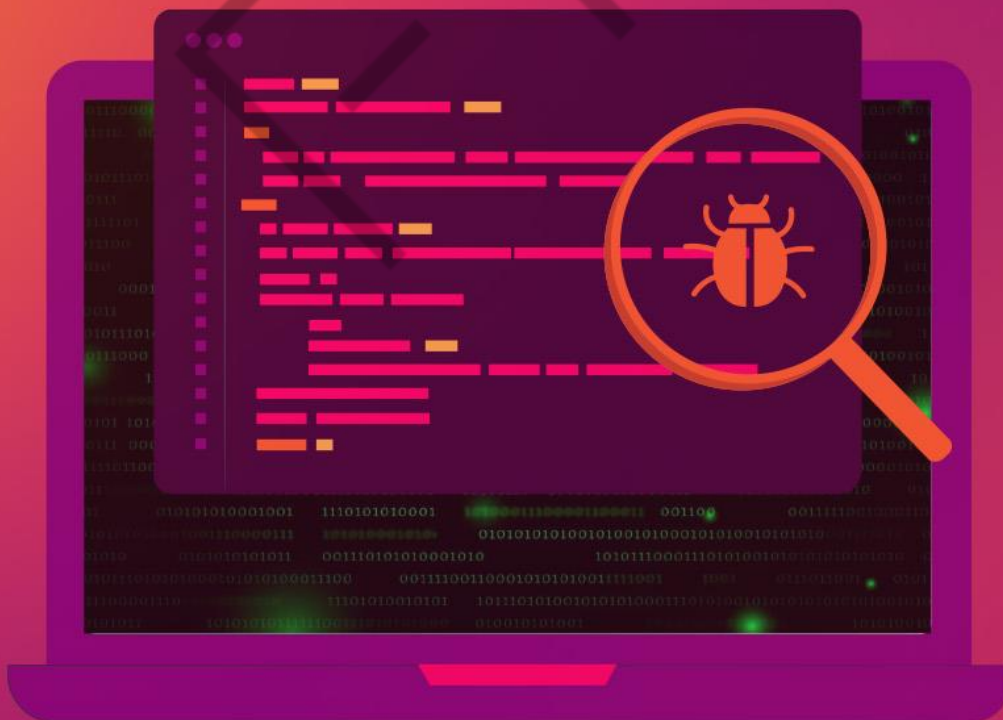


CSI CYBER

OFUSCAÇÃO DE MALWARES



6

LISTA DE FIGURAS

Figura 1 – Modificando o arquivo binário em disco	13
Figura 2 – Porta aberta pelo malware	15
Figura 3 – Verificação com AVG antivírus.....	16
Figura 4 – Abrindo ncx99.exe no LordPE.....	17
Figura 5 – Adicionando uma nova seção	18
Figura 6 – Editando a seção .NewSec	19
Figura 7 – Configurações da seção .NewSec	19
Figura 8 – Permitindo escrita na seção .text	20
Figura 9 – Inserção de bytes para seção .Fiap	21
Figura 10 – Salvando o <i>entry point</i>	22
Figura 11 – Salvando endereço de .text e .Fiap.....	23
Figura 12 – Código de <i>hijack</i> da execução	24
Figura 13 – Salvando alterações em disco (a)	24
Figura 14 – Salvando alterações em disco (b)	24
Figura 15 – Salvando o <i>entry point</i>	25
Figura 16 – Inserindo o código de codificação	27
Figura 17 – Salvando código no binário (a).....	28
Figura 18 – Salvando código no binário (b).....	28
Figura 19 – Seção .text sendo criptografada.....	29
Figura 20 – Configurando <i>breakpoint</i>	29
Figura 21 – Seção .text antes do encode	30
Figura 22 – Seção .text depois do encode	30
Figura 23 – <i>Bypass</i> no AVG	31

LISTA DE TABELAS

Tabela 1 – Tabela verdade do AND, OR e XOR	14
--	----

EMSE

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Código em C, ofuscação de variável	9
Código-fonte 2 – Código em C, código morto	10
Código-fonte 3 – Código em ASM de criptografia simples	26

EMANIP

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Portas abertas com netstat	15
--	----

EMANIP

SUMÁRIO

1 OFUSCAÇÃO DE MALWARE.....	7
1.1 Ofuscação	7
2 TÉCNICAS DE OFUSCAÇÃO.....	8
2.1 Ofuscação baseada em dados.....	8
2.2 Dobramento constante	9
3 INCLUSÃO DE CÓDIGO MORTO	10
4 SUBSTITUIÇÃO COM OPERADORES LÓGICOS	11
5 OFUSCAÇÃO BASEADA EM CONTROLE.....	11
6 OFUSCANDO UM MALWARE NA PRÁTICA	12
7 FLUXO DE EXECUÇÃO	13
8 POR QUE UTILIZAR O XOR (OU EXCLUSIVO)?	14
9 ARQUIVO INICIAL	15
10 LORDPE.....	17
10.1 XVI32.....	20
10.2 OllyDbg.....	21
CONCLUSÃO.....	31
REFERÊNCIAS.....	32

1 OFUSCAÇÃO DE MALWARE

O código gerado pelo compilador é extremamente complexo para fazer uma engenharia reversa, ficando ainda pior quando adicionado algum tipo de ofuscação.

Criadores de malwares se utilizam de *packing* e ofuscação para dificultar, primeiramente, a detecção do seu malware por softwares de antivírus e, conseqüentemente, o trabalho de um analista de *malware*.

Quando o autor de *malwares* ofusca seu *malware*, ele está tentando ocultar a execução desse *malware*. Os *malwares* ofuscados tendem a ter poucas *strings* e poucas chamadas de funções da API do Windows, por exemplo, a *LoadLibrary* e a *GetProcAddress*, estas funções são utilizadas para carregar novas funções.

Quando o *malware* compactado é executado, um pequeno programa *wrapper* também é executado para descompactar o arquivo compactado e, em seguida, executar o arquivo.

Neste capítulo, aprofundaremos mais na ofuscação de *malwares* e aprenderemos como ocorre o processo de ofuscação para que ele não seja detectado por um antivírus.

1.1 Ofuscação

Como vimos na introdução, reverter o código gerado pelo compilador, linguagem de máquina, para código de uma linguagem de baixo nível é um processo difícil e complexo e a situação piora quando implementamos uma ofuscação.

Essa técnica é muito utilizada por grandes empresas com o objetivo de, por exemplo, proteger chaves criptografadas, protocolos etc. Um bom exemplo é a *feature* DRM da Microsoft, tecnologia onde os proprietários de conteúdo podem proteger seus arquivos de mídia por meio de uma chave de criptografia que pode bloquear (deixar os dados ofuscados) e desbloquear o conteúdo.

Outro exemplo é a propriedade intelectual e sua proteção. Muitas empresas protegem seus softwares ofuscando-os para que não sejam replicados sem a devida autorização. Já outros sistemas empregam a ofuscação em outras áreas do

software, por exemplo, a ofuscação do protocolo de comunicação entre o cliente do Google Drive e os servidores.

Para o autor de *malwares*, utilizar destas técnicas de ofuscação serve para evitar e/ou dificultar a análise de mecanismos de detecção de antivírus, além da análise reversa do código do *malware* por analistas de *malware*.

Abstraindo um pouco mais, a ofuscação é apenas uma transformação de programa. O principal objetivo desse método é ter como entrada um programa e como saída a geração de um novo programa totalmente modificado e desigual e com a mesma equivalência computacional ou semântica do programa original, porém muito mais difícil de ser analisado.

O termo “difícil de ser analisado” é uma definição informal totalmente empírica. Por exemplo, imagine que uma operação básica de escrita em um arquivo possa ser executada pelo processador com aproximadamente 20 instruções de *assembly* e agora imagine que esta mesma operação possa ser executada com 10.000 instruções *assembly*. Qual dessas duas operações seria mais fácil de ser analisada?

No sentido mais geral, ofuscar um programa significa transformá-lo em uma forma que seja mais difícil para um adversário entender ou alterar o código original. Somos deliberadamente vagos sobre a definição de “difícil”, mas, normalmente entendemos que o programa ofuscado requer mais tempo humano, mais dinheiro ou mais capacidade de computação para analisar do que o programa original. Segundo essa definição, distribuir um programa em uma forma compilada, em vez de como código-fonte, é uma forma de ofuscação, já que analisar código de máquina binário é mais exigente do que ler o código-fonte. Da mesma forma, consideraríamos um programa que foi otimizado para ser mais ofuscado do que outro que não foi, já que muitas otimizações de código tornam a análise (tanto por humanos quanto por ferramentas, como disassemblers e decompilers) mais onerosa (tradução do autor) (COLLBERG & NAGRA, 2009).

2 TÉCNICAS DE OFUSCAÇÃO

2.1 Ofuscação baseada em dados

A ofuscação baseada em dados trata os dados isoladamente dentro das funções e tem efeito nos valores dos dados e cálculos sem controle. Serão

apresentadas algumas das principais características da ofuscação baseada em dados.

2.2 Dobramento constante

O Dobramento constante em um software de ofuscação consiste em: o software recebe um valor de entrada constante do programa original e aplica um ou mais processos computacionais para que se chegue ao mesmo valor da constante recebida, por exemplo:

Imagine que o *malware* tem o valor constante de entrada igual a 17 (o número da linha no código abaixo não reflete o código exato e serve apenas como referência).

```
01 x = 1156
02 y = 2
03 int painFunc(int x, int y) {
04     w = sqrt(x)
05     z = w/y
06     return z;
07 }
```

Código-fonte 1 – Código em C, ofuscação de variável
Fonte: Elaborado pelo autor (2020)

Como podemos observar, em nenhuma linha aparece o número 17, porém após a execução da função, o retorno será o número 17, acompanhe:

- Linha 01 e 02:
 - Atribuindo os valores 1156 e 2 às variáveis “x” e “y”, respectivamente;
- Linha 03:
 - É criada uma função que recebe como entrada essas duas variáveis;
- Linha 04:
 - sqrt calcula a raiz quadrada de “x” (1156), que é igual a 34 e armazena esse valor em “w”;
- Linha 05:
 - Agora “z” recebe o valor da divisão de “w” (34) por “y” (2) e a função retorna o valor de “z”, que será igual a 17.

Observe que este é um exemplo simples de dobramento constante de uma ofuscação, porém, quando esse código for analisado em linguagem *assembly* a simplicidade se torna algo um pouco mais complexo.

Se pensarmos de forma inversa, podemos imaginar que isso é uma das características mais básicas de otimização de um compilador. Por exemplo, imagine a seguinte declaração em código C:

```
x = 5 * 6;
```

O resultado da expressão é 30, pois os valores dos operandos e o operador (*) são estáticos. Com isso, o compilador pode simplesmente substituir a operação acima por `x = 30;` economizando, assim, instruções computacionais para o processador.

Outro exemplo seria:

- 01: push FE56FCB3h
- 02: add dword ptr [esp], 01A9034Dh

Esses 2 códigos em *assembly* podem ser equivalentes a um.

- 01: push 0h

É claro que desconsiderando as sinalizações das *flags*.

3 INCLUSÃO DE CÓDIGO MORTO

Uma das principais otimizações dos compiladores é a eliminação de código morto onde o compilador elimina toda e qualquer instrução que não afeta a operação do programa. Vejamos o exemplo abaixo:

```
01 int func1() {  
02     int x, y, z;  
03     x = 5;  
04     y = 3;  
05     z = 7;  
06     return z;  
07 }
```

Código-fonte 2 – Código em C, código morto
Fonte: Elaborado pelo autor (2020)

- Linha 01 e 02:
 - Inicialização da função `func1` e das variáveis;
- Linha 03 e 04:
 - Atribuição dos valores 5 e 3 às variáveis “x” e “y”, respectivamente. Aqui, temos um exemplo de código morto;
- Linha 05:
 - Atribuição do valor 7 à variável “z”;
- Linha 06:
 - Retorno do valor da função que será o valor atribuído à variável “z” igual a 7. Aqui, verificamos que as linhas 03 e 04 não possuem qualquer interferência na operação da função.

Os ofuscadores utilizam o inverso desta técnica dos compiladores para atribuir códigos sem sentido que não afetarão a operação do *malware*. Quando o analista de *malware* se depara com o *malware* ofuscado, terá que decidir manualmente quais instruções participam ou não das operações e resultados de cada uma.

4 SUBSTITUIÇÃO COM OPERADORES LÓGICOS

Declarações matemáticas podem ser realizadas associando os resultados de alguns operadores com resultado de combinações de outros operadores. Com isso, podemos inverter cada *bit* nos registradores ou diretamente em memória.

5 OFUSCAÇÃO BASEADA EM CONTROLE

A engenharia reversa se baseia na previsibilidade das traduções e controles de fluxo dos compiladores. Assim, os engenheiros reversos conseguem rapidamente estruturar o fluxo do código original em um nível de abstração acima da linguagem *assembly* software.

Por décadas, a engenharia reversa foi baseada em presunções de como os compiladores geravam os códigos de um determinado software. Para exemplificar, imagine um software qualquer compilado, em que todo o código estaria em um único bloco. Dessa forma, por meio da engenharia reversa, o código seria facilmente interpretado, utilizando a linguagem *assembly* e revertido para uma linguagem de alto nível como a linguagem C, por exemplo.

Instruções CALL corresponderão a chamadas de funções, assim como RET, normalmente, indicará um término de função. Ferramentas de análise estática são programadas para fazer tais correspondências anteriormente descobertas pelos humanos, a saber:

- CALL, utilizado para chamar funções e cada função se inicia nos endereços chamados pela instrução CALL.
 - Ex.: CALL 44562596, onde 44562596 é o endereço da função chamada.
- Após cada término de função, existe uma instrução RET que deverá retornar o fluxo da execução para o endereço subsequente à chamada CALL.

Essas são algumas premissas básicas dos compiladores que todo e qualquer software compilado deverá apresentar. A violação de tais premissas levará a um código de máquina pobre e muito difícil de se analisar estaticamente.

6 OFUSCANDO UM MALWARE NA PRÁTICA

Grande parte dos *softwares* de antivírus usa, em um primeiro momento, assinatura de código como tecnologia de varredura. Eles tentam identificar o *malware*, comparando-o com uma base de dados local que contém assinaturas de arquivos anteriormente reconhecidos. Caso o arquivo que esteja sendo analisado corresponda a esta assinatura, ele será sinalizado como arquivo malicioso.

Lembre-se: O software antivírus geralmente verifica arquivos no disco e não na memória.

Pense o seguinte: se alterássemos apenas 1 *byte* em determinado local no nosso *malware*, provavelmente, ele ficaria indetectável pelo antivírus, porém perderíamos sua funcionalidade ou até não seria executado normalmente.

Precisamos contornar esse obstáculo e fazer com que o *malware* não seja detectado pelo software de antivírus nem perca suas funcionalidades. Uma das soluções é manter o conteúdo do *malware* em disco totalmente codificado com uma chave e, quando este *malware* for executado, carregado em memória, seu conteúdo seja decodificado para o original.

7 FLUXO DE EXECUÇÃO

Após a execução do arquivo, seu conteúdo é carregado do disco para a memória principal, onde o processador começará a executar as instruções em código de máquina linha a linha. O ponto de entrada é alterado para que nosso código de decodificação seja executado primeiro e, após a decodificação, a operação do *malware* seja normalmente executada.

Repare abaixo a imagem que mostra as alterações feitas no arquivo original no disco.

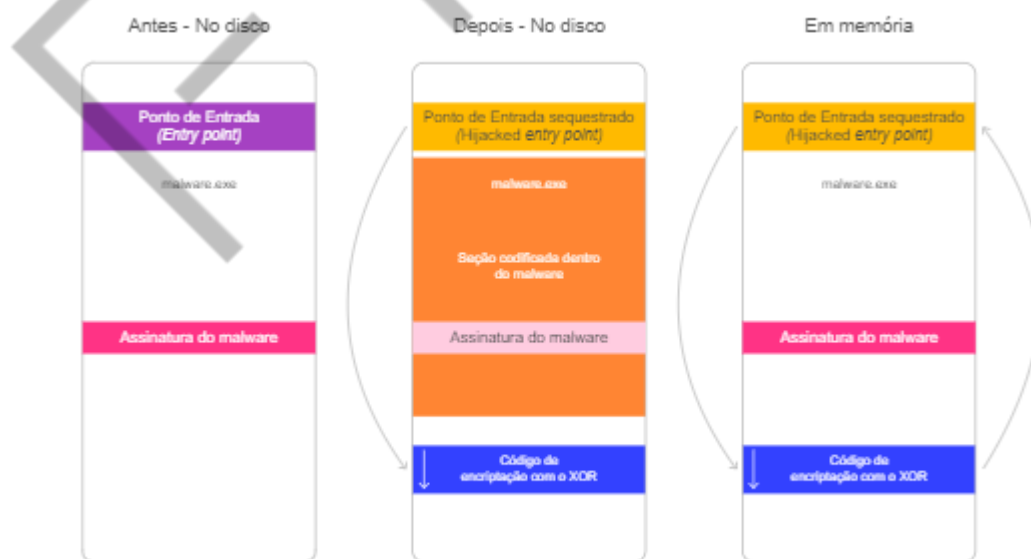


Figura 1 – Modificando o arquivo binário em disco
Fonte: Elaborado pelo autor (2020)

Como podemos observar, foi criado um salto no ponto de entrada no arquivo original que direciona o fluxo de execução para outra seção no código binário onde se localiza nosso código de decodificação. Então, este código será executado em memória para decodificar toda a seção do *malware*. Depois disso, voltará ao ponto de entrada (*entry point*) para a execução normal do *malware*.

O código de encriptação terá uma rotina XOR e receberá dois endereços: um inicial que começará a encriptação e outro final que irá terminá-la. Assim que a rotina XOR terminar de encriptar os dados, salvaremos todo o conteúdo em disco. Sendo ele um novo arquivo (ou um novo binário), sem uma assinatura conhecida de *malware*.

Quando este arquivo for executado e carregado na memória, o mesmo código de encriptação será executado com sua rotina XOR nos dados encriptados, voltando ao conteúdo original, mas, agora, em memória. Após todo o código decodificado, o código fará um salto para o ponto de entrada e continuará a execução normal do *malware*.

8 POR QUE UTILIZAR O XOR (OU EXCLUSIVO)?

Em criptografia, o XOR é amplamente utilizado por ter a capacidade de retornar aos dados originais, *bit a bit*, se aplicada a mesma chave aos dados criptografados, conhecida como cifra de chave única, ou seja, a operação é totalmente reversível.

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Tabela 1 – Tabela verdade do AND, OR e XOR
Fonte: Elaborado pelo autor (2020)

Como podemos observar na tabela verdade, o AND tem 75% de chance de ser o *bit* 0 e 25% de chance de ser o *bit* 1, que se inverte na tabela OR com 25% para ser o *bit* 0 e 75% para o *bit* 1. Com o XOR, a tabela nos mostra que temos 50%

de chance de retornar ou o *bit* 0 ou o *bit* 1. Com isso, os dados criptografados serão distribuídos uniformemente.

9 ARQUIVO INICIAL

Vejamos, inicialmente, qual é o objetivo principal do *malware* e analisaremos o arquivo com antivírus.

Executamos o comando abaixo antes e depois da execução do *malware* para verificar qual porta o *malware* abre.

```
C:\Windows\System32> netstat -pan TCP
```

Comando de prompt 1 – Portas abertas com netstat
Fonte: Elaborado pelo autor (2020)

Como podemos perceber na imagem abaixo, o *malware* abre a porta 99 para que o atacante possa se conectar remotamente ao alvo, ou seja, estamos lidando com um *backdoor*.

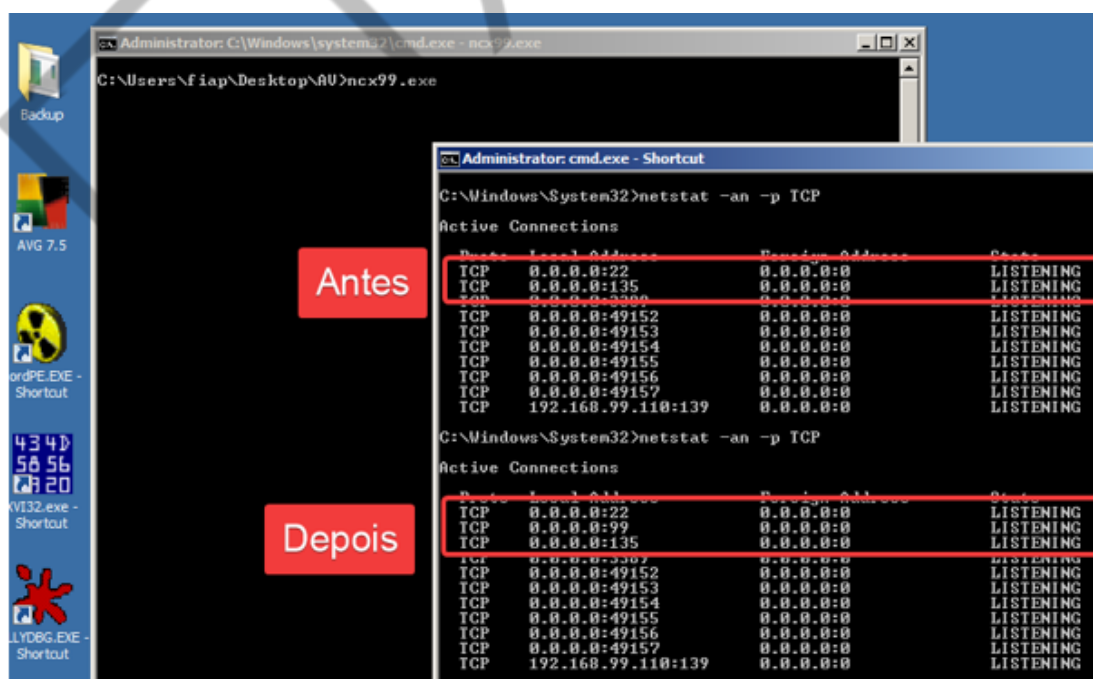


Figura 2 – Porta aberta pelo malware
Fonte: Elaborado pelo autor (2020)

Verificaremos se nosso *backdoor* é reconhecido pelo antivírus AVG.

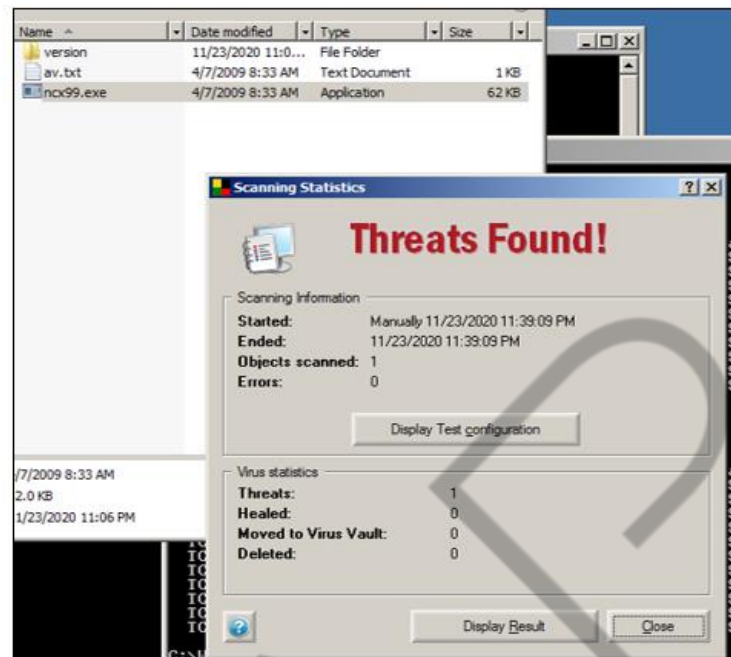


Figura 3 – Verificação com AVG antivírus
Fonte: Elaborado pelo autor (2020)

A partir de agora, trabalharemos diretamente com o arquivo binário, seguindo as seguintes etapas:

- LordPE;
 - Criação de uma nova seção dentro do binário chamada .Fiap;
 - Permitir a escrita na seção “.text” para que nosso código malicioso seja criptografado e salvo novamente dentro do binário.
- XVI32
 - Inserir 1.000 bytes dentro da nova seção criada anteriormente com o LordPE e salvar o arquivo como ncx99-1.exe.
- OllyDbg
 - Criar um salto para a seção .Fiap;
 - Criar o código que fará a criptografia dos dados da seção .text;
 - Executar todo o código para que a seção .text seja criptografada;
 - Salvar as alterações em disco;

- Certificar-se de que o *backdoor* está operando normalmente ao acessar a porta 99; e
- Realizar a verificação com antivírus.

10 LORDPE

Agora precisamos alterar as propriedades do nosso arquivo malicioso para que possamos injetar o código que executará a desofuscação em memória e permitirá o modo de escrita, onde se encontra o *backdoor*.

Criando a seção .Fiap dentro do arquivo binário (local onde será inserido nosso código de desofuscação/decriptografia/decode).

Abrir arquivo ncx99.exe no LordPE.

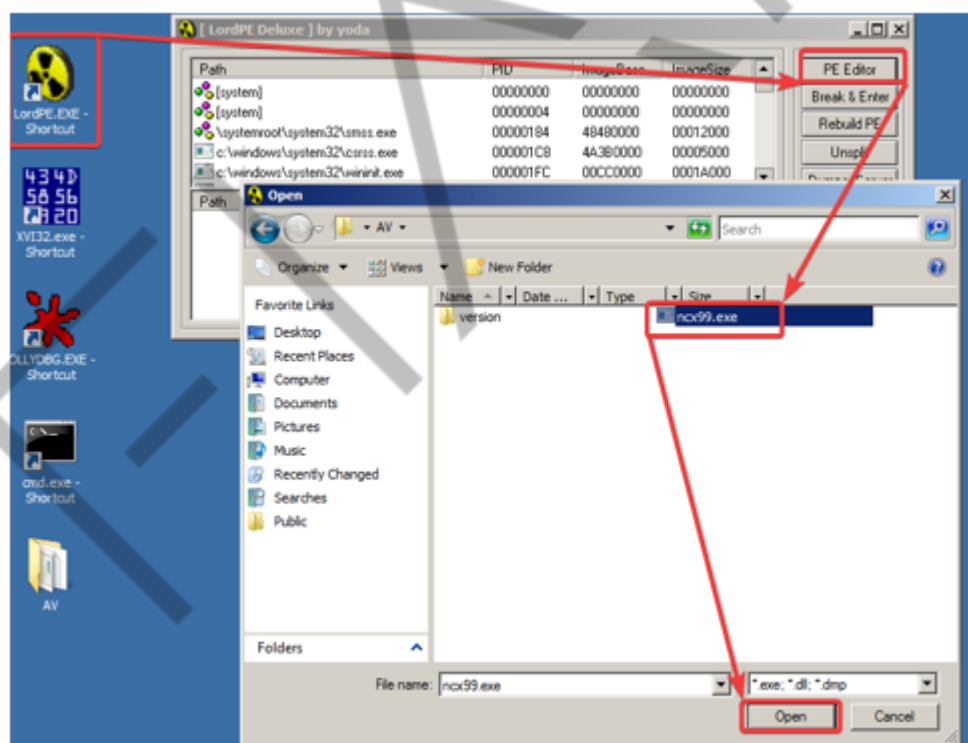


Figura 4 – Abrindo ncx99.exe no LordPE
Fonte: Elaborado pelo autor (2020)

Após o arquivo aberto, clique em seção e a tabela de seções do binário será mostrada. Para criar uma nova seção no arquivo binário ncx99.exe, primeiro, clique com o botão direito do mouse em qualquer seção e depois selecione “*add section header*”, como na imagem abaixo.

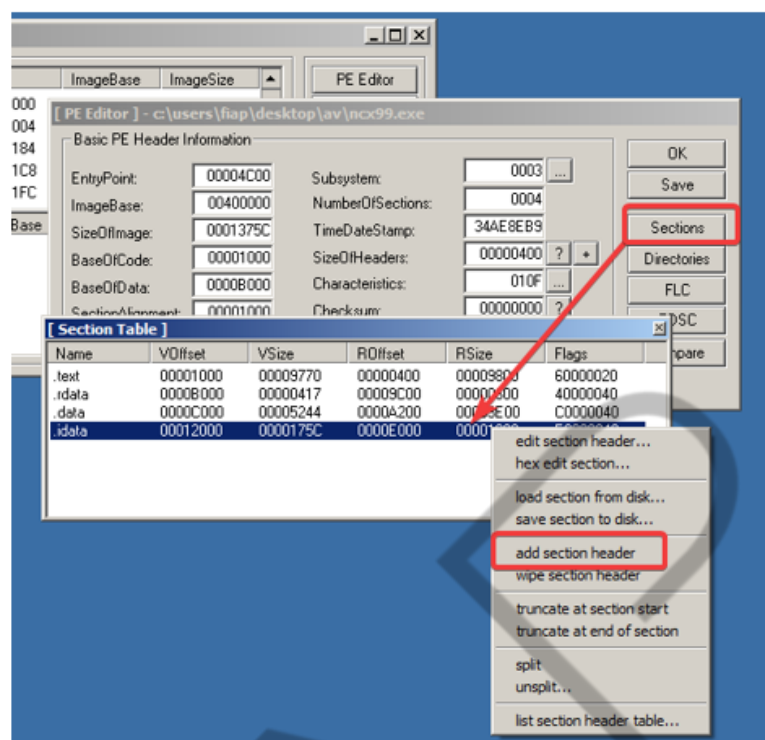


Figura 5 – Adicionando uma nova seção
Fonte: Elaborado pelo autor (2020)

Agora que a seção `.NewSec` foi criada, devemos renomear para `.Fiap` e configurar para que ela tenha o tamanho de 1.000 bytes. Depois disso, devemos abrir a seção `.text` e permitir o modo de escrita dentro desta seção, que, por padrão, está proibido.

O nosso *backdoor* está inserido em `.text`, é esta seção que será decodificada em memória, mas, primeiro, precisamos codificá-la dentro do arquivo. Por isso, o modo escrita está habilitado, quando todo *backdoor* estiver encriptado/ofuscado em disco, salvaremos esta modificação em disco.

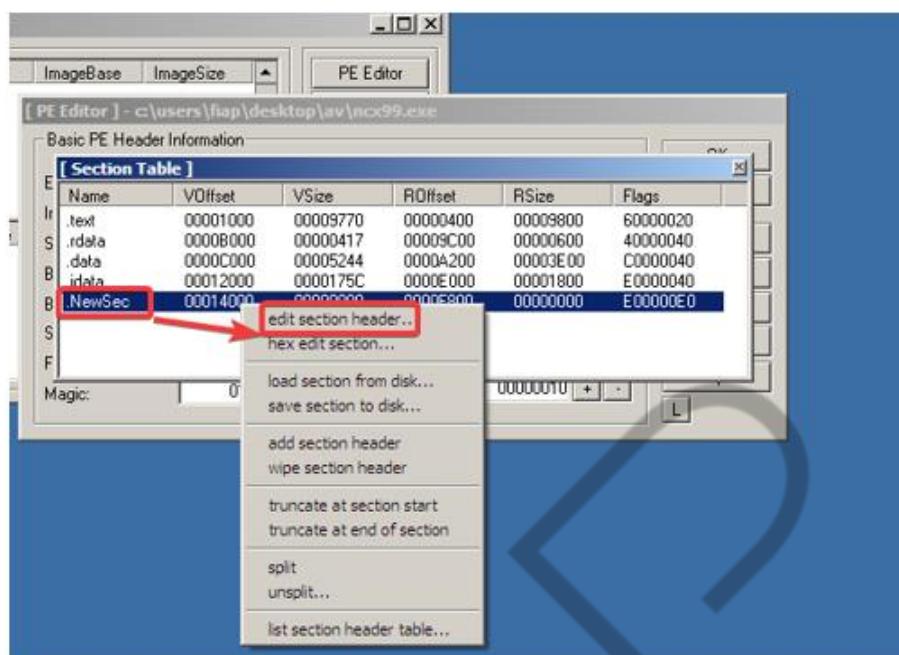


Figura 6 – Editando a seção .NewSec
Fonte: Elaborado pelo autor (2020)

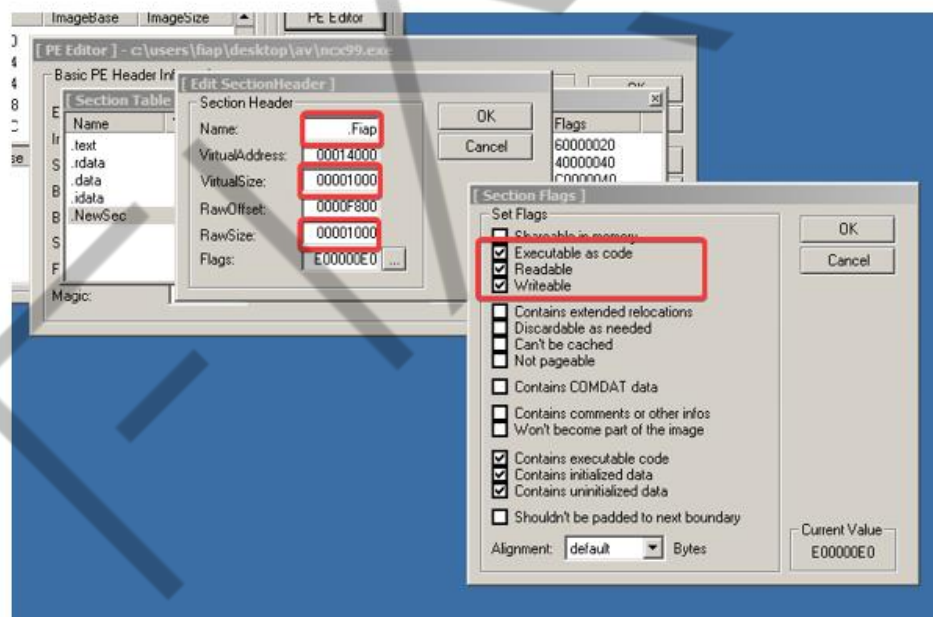


Figura 7 – Configurações da seção .NewSec
Fonte: Elaborado pelo autor (2020)

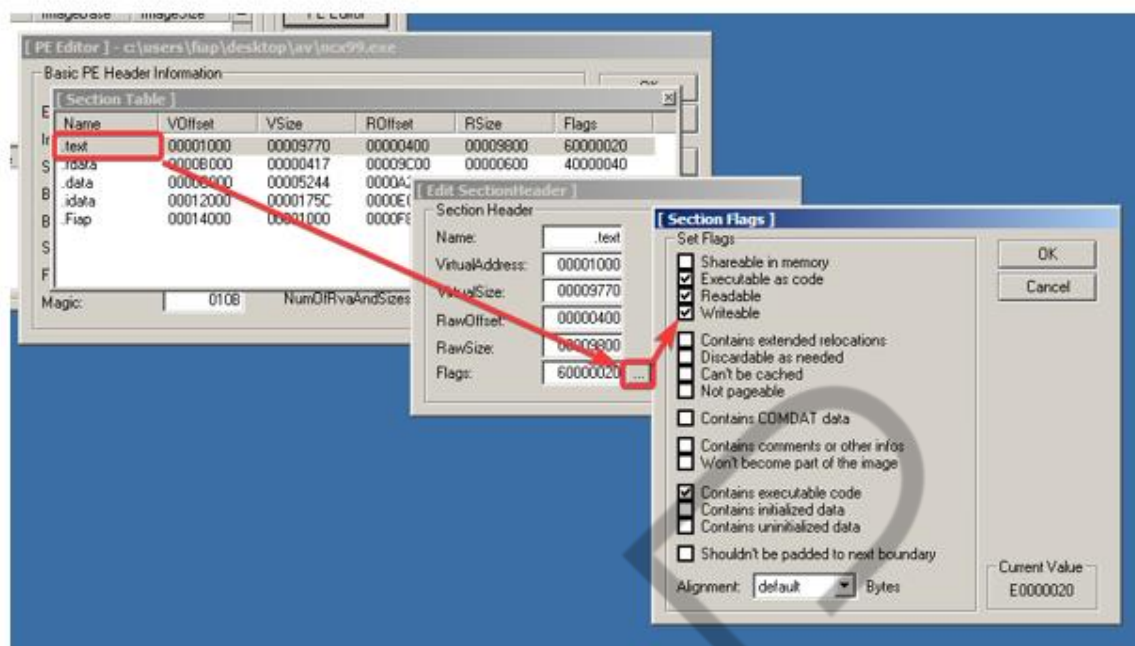


Figura 8 – Permitindo escrita na seção .text
Fonte: Elaborado pelo autor (2020)

Após pressionar “OK”, voltaremos até a janela inicial do LordPE e salvaremos o arquivo. Note que este arquivo não está funcional. Após executá-lo, você verá uma resposta de erro informando que o arquivo não é uma aplicação win32 válida.

10.1 XVI32

O problema é que criamos um cabeçalho para a nova seção e precisamos inserir os 1.000h bytes físicos dentro do arquivo. Para isso, utilizaremos um editor de hexadecimal, como o XVI32.

Abra o nosso arquivo ncx99.exe no XVI32 e selecione o último byte. Depois, clique em *Edit* → *Insert String*. Nesta janela, inserir “00” para o campo *Hex string* e “\$1000” para o campo hexadecimal, como a figura abaixo.

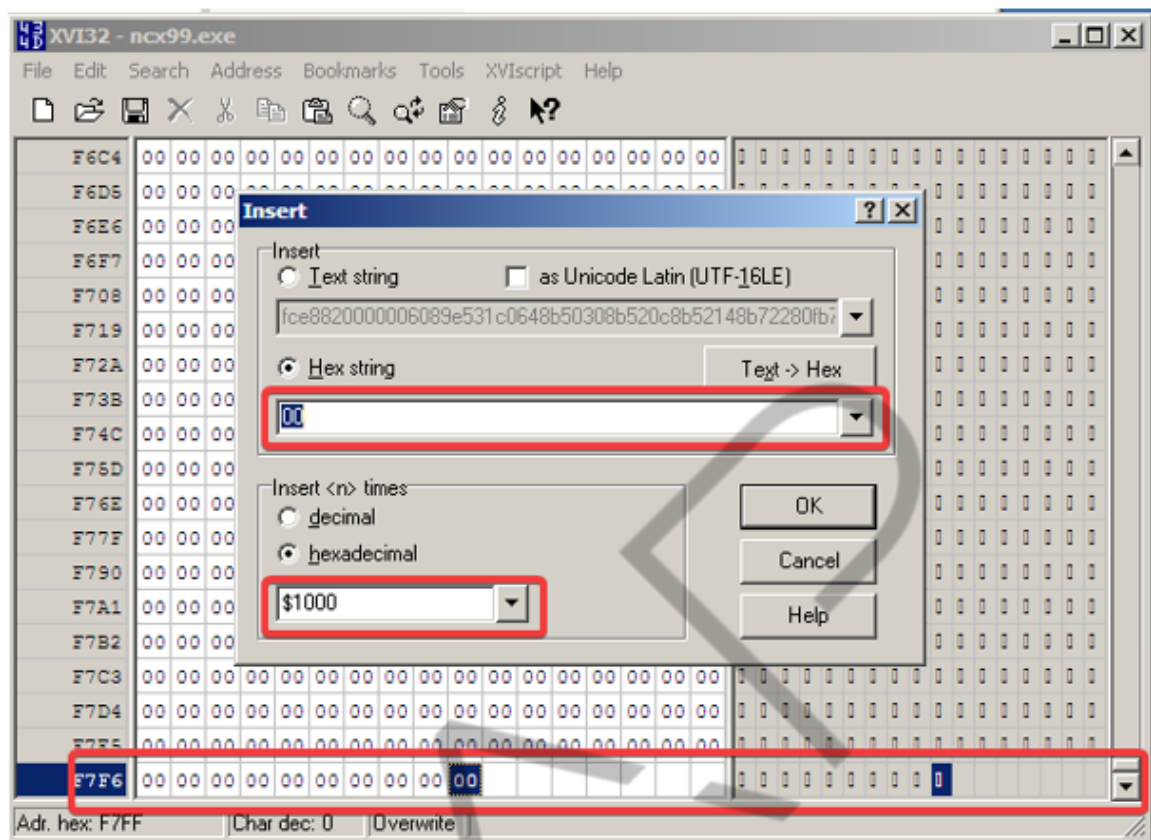


Figura 9 – Inserção de bytes para seção .Fiap
Fonte: Elaborado pelo autor (2020)

Após salvar o arquivo, tente executá-lo novamente. Como podemos observar, está executando sem erros. Agora o arquivo está pronto para a inserção do código que desofuscará o *backdoor* em memória. Para alcançar esse objetivo, utilizaremos o OllyDbg.

10.2 OllyDbg

Com o OllyDBG, editaremos o arquivo binário em *assembly*, apesar de serem comandos básicos, requerem maior atenção.

Após abrir o arquivo ncx99.exe no OllyDBG, o cursor estará no *entry point*. Devemos anotar esses endereços em um bloco de notas, pois faremos um *Hijacking Execution Flow* ou sequestro de execução do binário.

“Ok, Hugo, bonito nome, mas poderia explicar?”

Sim, claro, caro aluno! Lembra que adicionamos uma nova seção (.Fiap) ao binário? É lá que será inserido nosso código, que fará a criptografia da seção .text

(seção que contém o código de *backdoor*). Porém, precisamos alterar o fluxo de execução do binário a fim de que nosso código de descryptografia seja executado primeiro para que, depois, o fluxo de execução continue de onde foi sequestrado.

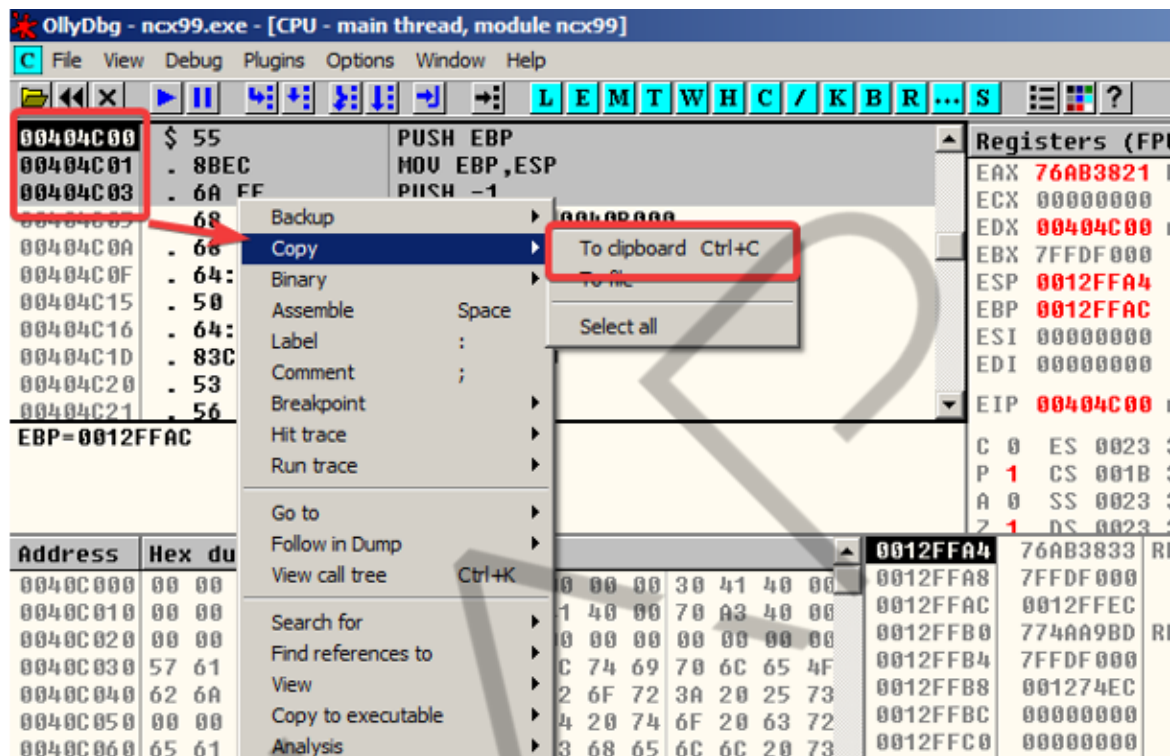


Figura 10 – Salvando o *entry point*
Fonte: Elaborado pelo autor (2020)

Agora devemos anotar, em um bloco de notas, os valores dos endereços de memória das seções “.text” e “.Fiap”. Para isso, clique na opção “M” do Olly, que exibirá uma janela com os endereços de memória das seções, como na figura: “Salvando endereço de .text e .Fiap”.

Address	Size	Owner	Section	Contains	Type	Access
00010000	00010000				Map	RW
00120000	00001000				Priv	RW
0012E000	00002000			stack of ma	Priv	RW
00130000	00004000				Map	R
00220000	00003000				Priv	RW
00250000	00004000				Priv	RW
00400000	00001000	ncx99		PE header	Imag	R
00401000	0000A000	ncx99	.text	code	Imag	R
0040B000	00001000	ncx99	.rdata		Imag	R
0040C000	00006000	ncx99	.data	data	Imag	R
00412000	00002000	ncx99	.idata	imports	Imag	R
00414000	00001000	ncx99	.Fiap		Imag	R
00420000	0007E000				Map	R
00850000	00001000				Priv	RW
74220000	00001000	WSOCK32		PE header	Imag	R

Figura 11 – Salvando endereço de .text e .Fiap

Fonte: Elaborado pelo autor (2020)

Após salvar todos os endereços no bloco de notas, vamos fazer efetivamente o *hijack* do fluxo de execução. Repare na imagem abaixo.

Primeiro, altere a janela para mostrar as instruções da  CPU.

Agora, altere a primeira instrução para o código abaixo. Apenas selecione o primeiro endereço e pressione a tecla de espaço. Será aberta uma janela para a inserção da instrução desejada, como na figura abaixo. Neste código, instruímos o processador a saltar (JMP - *jump*) para o endereço de memória 00414000, que é o início da seção .Fiap que criamos anteriormente.

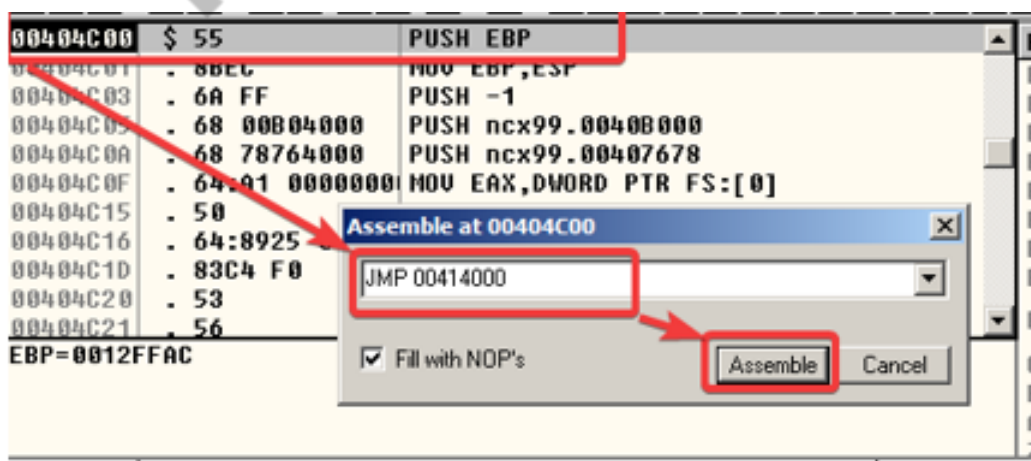


Figura 12 – Código de *hijack* da execução
Fonte: Elaborado pelo autor (2020)

Agora, devemos salvar nossas alterações em um novo arquivo, *ncx99-1.exe*, e reabri-lo no OllyDBG. Após aberto, pressione a tecla F7 para que o salto seja executado e alcance o endereço inicial de memória da seção *.Fiap*, como na imagem abaixo.

Para salvar as alterações siga sempre estes passos abaixo. Selecionar todas as alterações, clicar com o botão direito do mouse, como na Figura “Salvando alterações em disco (a)”. Na próxima janela, clicar novamente com o botão direito do mouse na seleção “*Save file*” e escolher um novo nome para o arquivo que está sendo salvo (*ncx99-1.exe*). Manteremos um simples versionamento para voltar um passo anterior caso necessário.

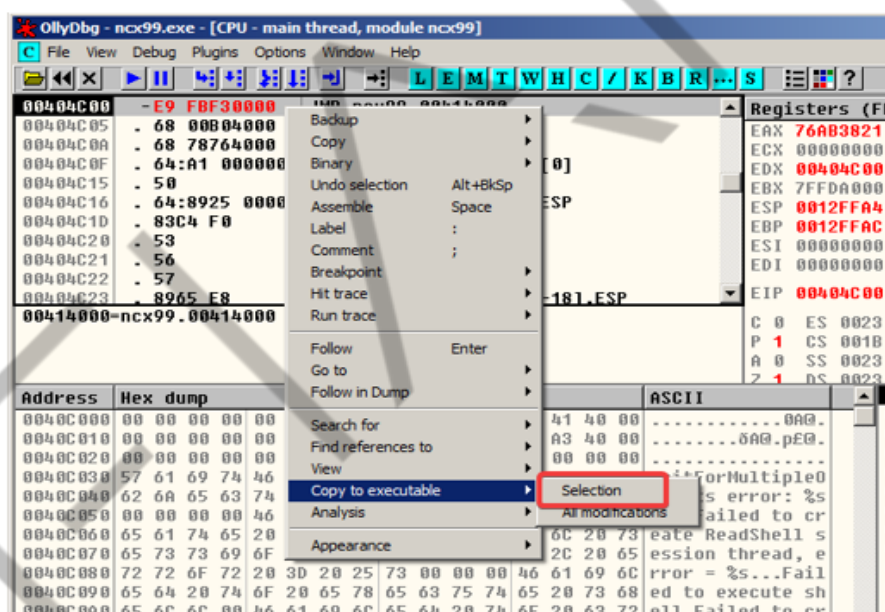


Figura 13 – Salvando alterações em disco (a)
Fonte: Elaborado pelo autor (2020)

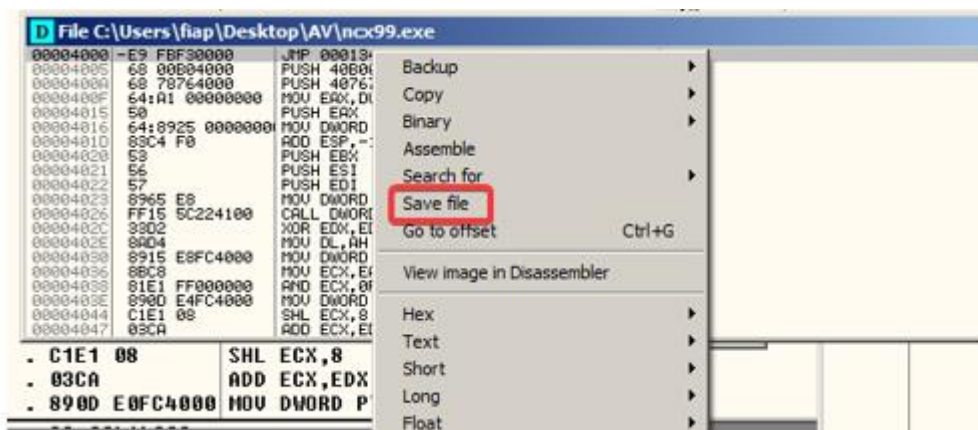


Figura 14 – Salvando alterações em disco (b)

Fonte: Elaborado pelo autor (2020)

Após o arquivo salvo e renomeado, abra o novo arquivo com o OllyDBG e pressione F7. Deverá chegar no endereço de memória 00414000, como na imagem abaixo:

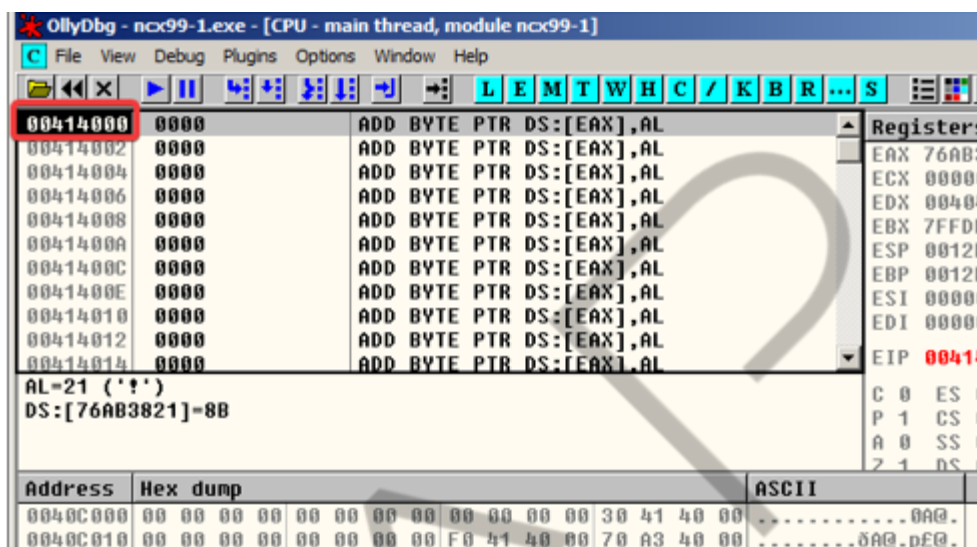


Figura 15 – Salvando o *entry point*
Fonte: Elaborado pelo autor (2020)

Após o F7 pressionado, podemos ver que tudo ocorreu como esperado.

A partir daqui, escreveremos nosso código que criptografa a seção *.text* em disco e, consequentemente, fará a descriptografia em memória, ou seja, quando a aplicação for executada. Não podemos simplesmente sair criptografando todo o código.

Dados importantes, como a tabela de importação, são essenciais para a execução do arquivo. Para este simples exemplo, codificaremos apenas a seção *.text*. Porém, nem sempre será o suficiente para se evadir completamente dos antivírus, mas é um começo.

Vamos ao código. Não se intimide com o código em *assembly*, explicarei cada linha e será de fácil entendimento. Resumindo, a base deste código *assembly* é apenas um *loop*. Isso mesmo, será executado um *loop* entre um endereço inicial e um final. Dentro deste *loop*, faremos um XOR em todos os bytes que estão compreendidos entre o endereço inicial e o endereço final.

Após a criptografia de todos os bytes, salvaremos o novo arquivo em disco (*loop* e os dados criptografados). Quando o arquivo for executado, o *loop* será

executado novamente e os dados que estavam criptografados em disco serão decriptografados em memória e a execução do *malware* ocorrerá naturalmente. Leia com atenção nosso código de criptografia:

```
01 00414000  MOV EAX,00404C05
02 00414005  XOR BYTE PTR DS:[EAX],0F
03 00414008  INC EAX
04 00414009  CMP EAX,0040A76F
05 0041400E  JLE SHORT 00414005
06 00414010  PUSH EBP
07 00414011  MOV EBP,ESP
08 00414013  PUSH -1
09 00414015  JMP 00404C05
```

Código-fonte 3 – Código em ASM de criptografia simples
Fonte: Elaborado pelo autor (2020)

- Linha 01:
 - Aqui movemos o endereço de memória 00404C05 para o registrador EAX. Nosso *loop* iniciará neste endereço que está em EAX. Lembre-se que este é nosso endereço inicial para começar a codificação/criptografia;
- Linha 02:
 - Executa o XOR no conteúdo que se encontra no primeiro endereço. Neste caso, 00404C05. Observe que a chave utilizada para o XOR é a 0F (seria possível também criar outros arquivos distintos do mesmo *malware* apenas alterando a chave. Temos 254 possibilidades, de 01 – FF);
- Linha 03:
 - Incrementa em 1 byte nosso EAX que agora passa a ser 00404C06;
- Linha 04:
 - Compara se o endereço de memória em EAX é igual a 0040A76F, ou seja, se já chegamos ao endereço final a ser codificado;
- Linha 05:
 - Faz um *jump* caso a última instrução seja falsa, ou seja, volta ao endereço do XOR e inicia novamente o *loop*;

- Linha 06 a 08:
 - Restaura as instruções que foram sequestradas (*hijack*) antes das modificações;
- Linha 09:
 - Volta o fluxo de execução normal do *malware*.

Agora devemos copiar nosso código dentro do binário ncx99-1.exe. Abra-o com OllyDBG e pressione F7. Depois disso, pressione a tecla espaço para editar as instruções. A cada instrução inserida, clique em *Assemble* e o OllyDBG passará automaticamente para a próxima instrução, bastando apenas colar a próxima instrução na janela. Ao final, teremos esta sequência, como na imagem abaixo.

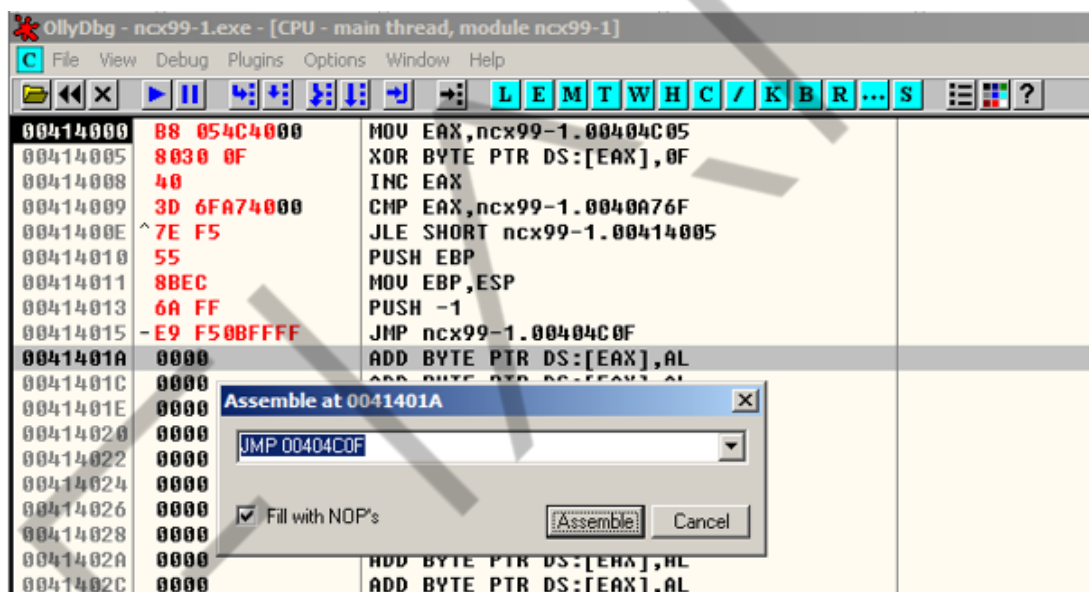


Figura 16 – Inserindo o código de codificação
Fonte: Elaborado pelo autor (2020)

Depois, devemos selecionar essas modificações e salvá-las em um novo binário, ncx99-2.exe.

Primeiro, selecione as instruções que foram alteradas, clique com o botão direito do mouse e selecione *Copy to executable* → *Selection*.

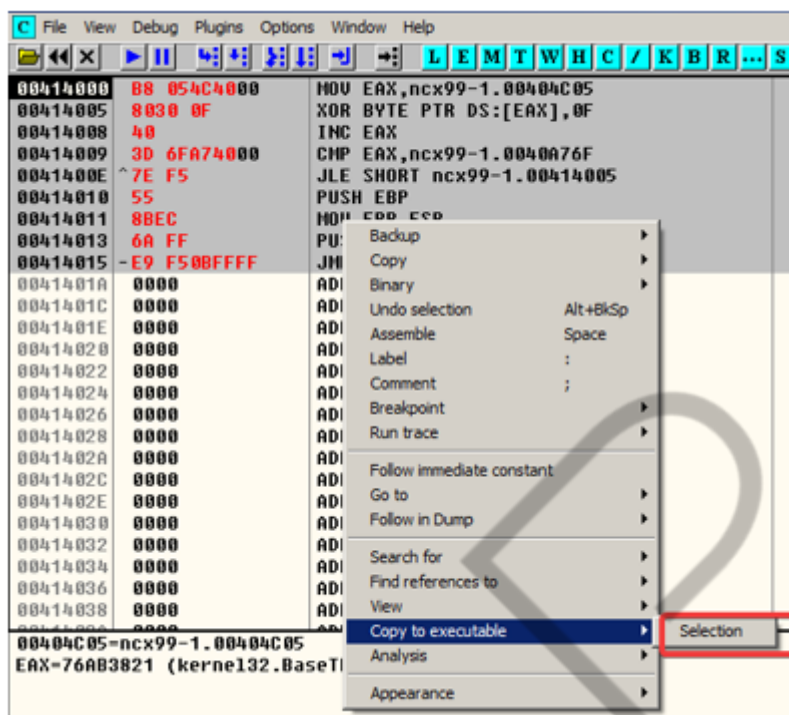


Figura 17 – Salvando código no binário (a)
Fonte: Elaborado pelo autor (2020)

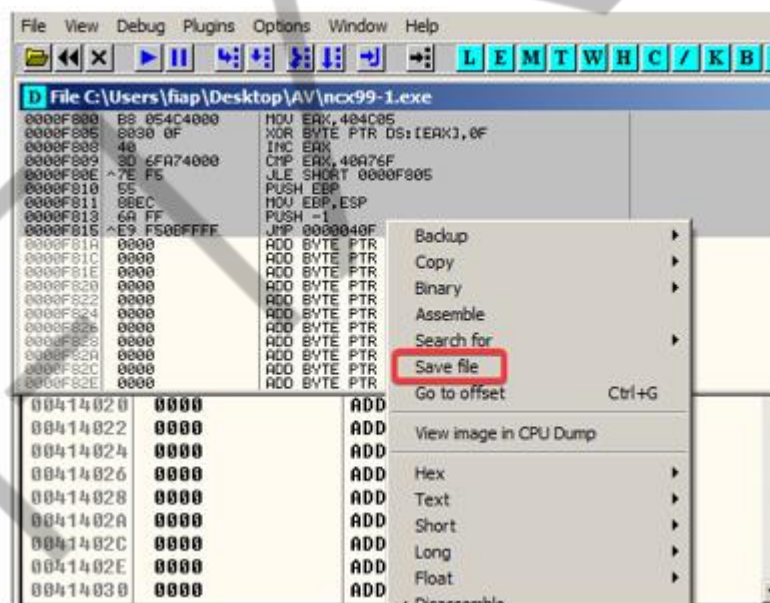


Figura 18 – Salvando código no binário (b)
Fonte: Elaborado pelo autor (2020)

Para mantermos o versionamento, salvaremos com o nome de ncx99-2.exe.

Agora abriremos o novo arquivo para que possamos criptografar todos os endereços que revelam a assinatura do *malware*. Após abrir o arquivo, pressione F7 e estaremos novamente no início da nossa seção .fiap na primeira instrução, para

que possamos ver nosso código codificando todos os bytes, siga as instruções abaixo.

Na janela de Dump da memória do OllyDBG, tecla Ctrl + G e digite o endereço inicial que será salvo em EAX 00404C05.

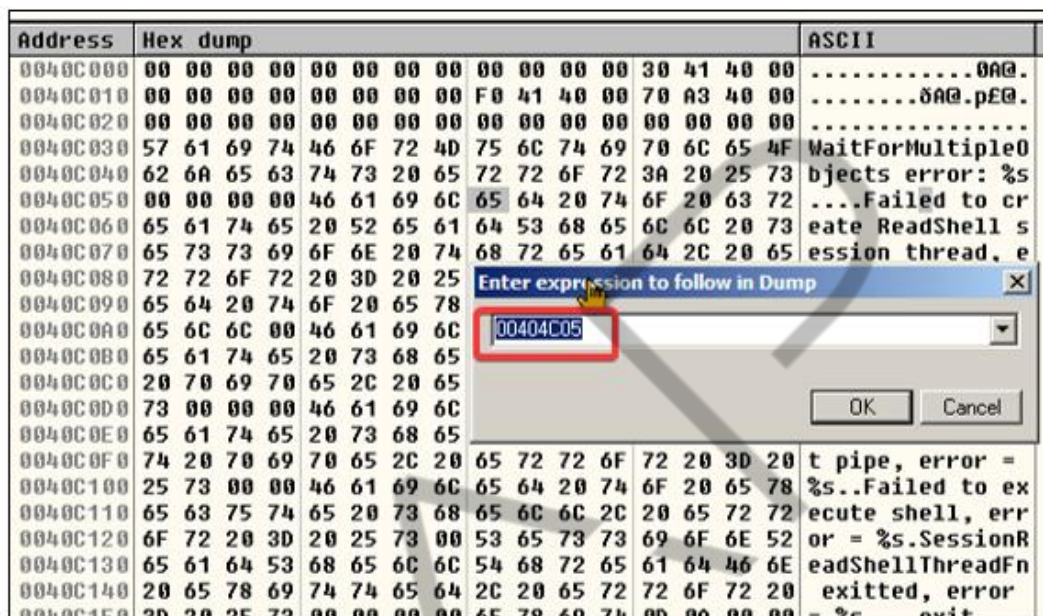


Figura 19 – Seção .text sendo criptografada

Fonte: Elaborado pelo autor (2020)

Agora continue pressionando a tecla F7 e a “mágica” acontecerá a cada incremento do EAX. Observe na janela de DUMP que os bytes vão se alterando um a um. Para agilizar o processo, configure um *breakpoint* no endereço 00414010 (selecione o endereço e tecla F2) e pressione a tecla F9 para que todos os endereços sejam criptografados em disco.

00414000	B8 054C4000	MOV EAX,ncx99-2.00404C05
00414005	8030 0F	XOR BYTE PTR DS:[EAX],0F
00414008	40	INC EAX
00414009	3D 6FA74000	CMP EAX,ncx99-2.0040A76F
0041400E	7E F5	JLE SHORT ncx99-2.00414005
00414010	55	PUSH EBP
00414011	8BEC	MOV EBP,ESP
00414013	6A FF	PUSH -1
00414015	E9 F50BFFFF	JMP ncx99-2.00404C0F
0041401A	0000	ADD BYTE PTR DS:[EAX],AL
0041401C	0000	ADD BYTE PTR DS:[EAX],AL
0041401E	0000	ADD BYTE PTR DS:[EAX],AL
00414020	0000	ADD BYTE PTR DS:[EAX],AL
00414022	0000	ADD BYTE PTR DS:[EAX],AL
00414024	0000	ADD BYTE PTR DS:[EAX],AL

Figura 20 – Configurando breakpoint

Fonte: Elaborado pelo autor (2020)

Address	Hex dump	ASCII
00404C05	68 00 B0 40 00 68 78 76 40 00 64 A1 00 00 00 00	h."@.hxx@.d;....
00404C15	50 64 89 25 00 00 00 00 83 C4 F0 53 56 57 89 65	Pd%....ÑSUVWe
00404C25	E8 FF 15 5C 22 41 00 33 D2 8A D4 89 15 E8 FC 40	èÿq\'A.30000èü@
00404C35	00 8B C8 81 E1 FF 00 00 00 89 0D E4 FC 40 00 C1	.ÑÉáÿ...Ñ.àü@.Á
00404C45	E1 08 03 CA 89 0D E0 FC 40 00 C1 E8 10 A3 DC FC	á00EÑ.àü@.Áèÿÿü
00404C55	40 00 E8 F4 00 00 00 85 C0 75 0A 6A 1C E8 B9 00	@.èô...ÑAu.jè'.
00404C65	00 00 83 C4 04 C7 45 FC 00 00 00 00 E8 EA 14 00	..Ñ0ÇEü....èè@.
00404C75	00 E8 E5 29 00 00 B8 7F 27 41 00 90 A3 28 02 41	.èã)...'A.É(0A
00404C85	00 E8 A5 25 00 00 A3 38 FD 40 00 85 C0 74 09 A1	.è¥%...É8ÿ@.ÑAt.;
00404C95	28 02 41 00 85 C0 75 0A 6A FF E8 BC F7 FF FF 83	(0A.ÑAu.jÿè%ÿÿÑ
00404CA5	C4 04 E8 D4 22 00 00 E8 DF 21 00 00 E8 7A F7 FF	Ñè0"...è0!...èzÿÿ
00404CB5	FF A1 F8 FC 40 00 A3 FC FC 40 00 50 A1 F0 FC 40	ÿ;@ü@.Éüü@.P;ðü@
00404CC5	00 50 8B 0D EC FC 40 00 51 E8 2D E8 FF FF 83 C4	.PÑ.üü@.Qè-èÿÿÑÑ
00404CD5	0C 89 45 E4 50 E8 81 F7 FF FF EB 21 8B 45 EC 8B	.ÑEäPè÷ÿÿè!ÑEíÑ
00404CE5	10 8B 0A 89 4D E0 50 51 E8 0E 20 00 00 83 C4 08	ÑÑ.ÑMàPQè...ÑÑ
00404CF5	C3 8B 65 E8 8B 45 E0 50 E8 7E F7 FF FF 83 C4 04	ÑÑeèÑEäPè~÷ÿÿÑÑ
00404D05	C7 45 FC FF FF FF FF 8B 4D F0 64 89 0D 00 00 00	ÇEüÿÿÿÿÑMððÑ....
00404D15	00 5F 5E 5B 8B E5 5D C3 90 90 90 83 3D 40 FD 40	._^[Ñã]ÑÑ=áÿ@
00404D25	00 02 74 05 E8 22 2A 00 00 8B 44 24 04 50 E8 58	tt0è"*...ÑÑÑPäx

Figura 21 – Seção .text antes do encode

Fonte: Elaborado pelo autor (2020)

Address	Hex dump	ASCII
00404C05	67 0F BF 4F 0F 67 77 79 4F 0F 6B AE 0F 0F 0F 0F	g0;00gwy00k@0000
00404C15	5F 6B 86 2A 0F 0F 0F 0F 8C CB FF 5C 59 58 86 6A	k*0000Éÿ\YXÑj
00404C25	E7 F0 1A 53 2D 4E 0F 3C DD 85 DB 86 1A E7 F3 4F	çð0S-Ñ0<ÿ000ç00
00404C35	0F 84 C7 8E EE F0 0F 0F 0F 86 02 EB F3 4F 0F CE	0ÑÇÑîð000Ñè000î
00404C45	EE 07 0C C5 86 02 EF F3 4F 0F CE E7 1F AC D3 F3	î0.ÑÑ0í000îç-00
00404C55	4F 0F E7 FB 0F 0F 0F 8A CF 7A 05 65 13 E7 B6 0F	00ç0000Ñÿz00ç00
00404C65	0F 0F 8C CB 0F C8 4A F3 0F 0F 0F 0F E7 E5 1B 0F	00ÑÈ0J0000ç00
00404C75	0F E7 EA 26 0F 0F B7 70 28 4E 0F 9F AC 27 0D 4E	0çè00-p(Ñ0Ñ-' .N
00404C85	0F E7 AA 2A 0F 0F AC 37 F2 4F 0F 8A CF 7B 06 AE	0çª*00-7ð00Ñÿ{00
00404C95	27 0D 4E 0F 8A CF 7A 05 65 F0 E7 B3 F8 F0 F0 8C	' .NÑÿz00èç³00ðÑ
00404CA5	CB 0B E7 DB 2D 0F 0F E7 D0 2E 0F 0F E7 75 F8 F0	È0ç0-00ç0.00ç00ð
00404CB5	F0 AE F7 F3 4F 0F AC F3 F3 4F 0F 5F AE FF F3 4F	ð0÷000-0000_0ÿ00
00404CC5	0F 5F 84 02 E3 F3 4F 0F 5E E7 22 E7 F0 F0 8C CB	0Ñð000^ç"çððÑÈ
00404CD5	03 86 4A EB 5F E7 8E F8 F0 F0 E4 2E 84 4A E3 84	0ÑJè çÑðððð.ÑJã
00404CE5	1F 84 05 86 42 EF 5F 5E E7 01 2F 0F 0F 8C CB 07	0ÑBÿ_ç0/00ÑÈ0
00404CF5	CC 84 6A E7 84 4A EF 5F E7 71 F8 F0 F0 8C CB 0B	îjçÑJÿ ç00ððÑÈ
00404D05	C8 4A F3 F0 F0 F0 F0 84 42 FF 6B 86 02 0F 0F 0F	ÈJ0ððððÑBÿk0000
00404D15	0F 50 51 54 84 EA 52 C0 9F 9F 9F 8C 32 4F F2 4F	0PQTÑBÿ000020ð0
00404D25	0F 0D 7B 0A E7 2D 25 0F 0F 84 4B 2B 0B 5F E7 57	0-ÿ.ç000ÑK+0_çW
00404D35	25 0F 0F 8C CB 0B 67 F0 0F 0F 0F F0 1A CF DE 4F	%00ÑÈ0ð000ð0ÿ0

Figura 22 – Seção .text depois do encode

Fonte: Elaborado pelo autor (2020)

Com toda a assinatura do *malware* criptografada, podemos salvar em um novo arquivo seguindo as etapas a seguir:

Selecione todo o conteúdo entre os endereços 00404C05 e 0040A76F e salve-o em um novo arquivo ncx99-3.exe. Após executá-lo, verifique se a porta 99 foi aberta corretamente com o comando netstat. Depois, execute a verificação com o antivírus AVG. Se tudo estiver certo, o AVG não conseguirá detectar nenhuma ameaça.

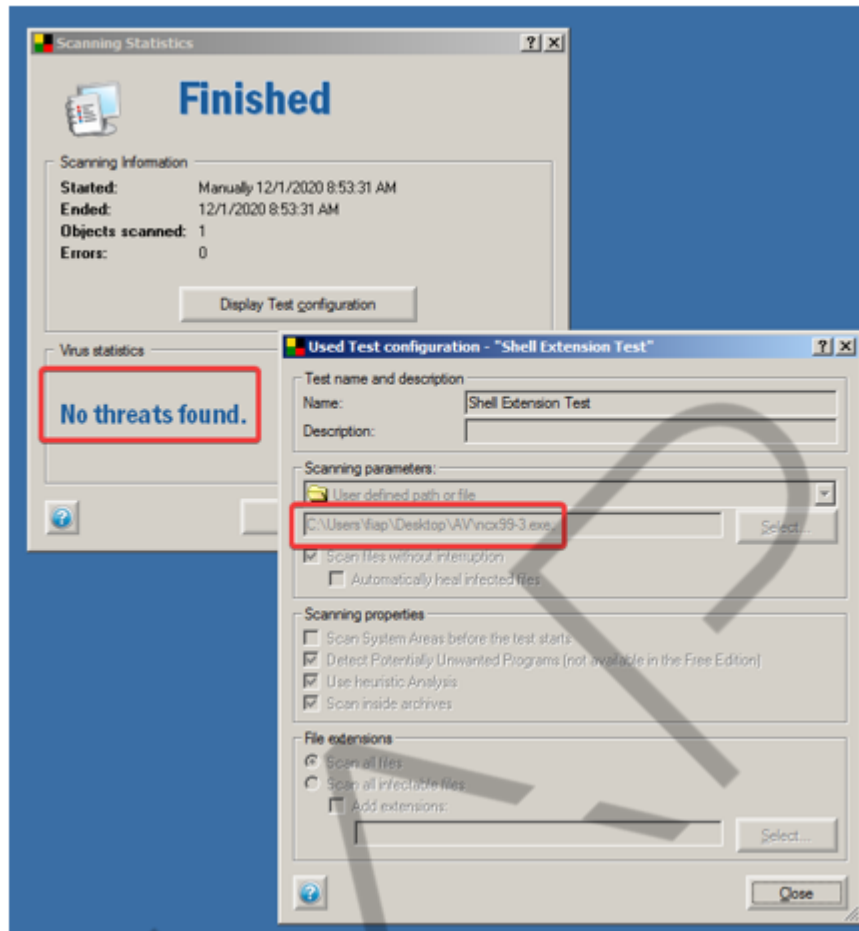


Figura 23 – Bypass no AVG
Fonte: Elaborado pelo autor (2020)

CONCLUSÃO

Embora as técnicas para a ofuscação de dados sejam amplamente utilizadas por desenvolvedores de software para tornar a engenharia reversa de suas aplicações mais difícil, os autores de *malwares* se beneficiam destas técnicas para ofuscar seus *malwares* e, conseqüentemente, torná-los mais difíceis de serem detectados.

Apesar de simples, a técnica apresentada é muito utilizada nos dias de hoje. Com algumas modificações, pode-se passar antivírus atualizados. É de grande importância que o aluno repita todos os passos para se familiarizar com o OlyDBG e com a linguagem *assembly*.

REFERÊNCIAS

COLLBERG, C.; NAGRA, J. **Surreptitious Software**: Obfuscation, Watermarking, and Tamperproofing for Software Protection. [S.l.]: Addison-Wesley Professional, 2009.

EMASP